ARTE DIGITAL. CONSISTÊNCIA TÉCNICA.

Por dentro da solução Kurio

Arquitetura, regras de negócio e roteiro para a entrevista.



Este guia acompanha o código e suas evidências. Use-o para explicar decisões e demonstrar comportamentos observáveis, incluindo falhas e limites reais.

Stack: React, TypeScript, TanStack Router, TanStack Query, Axios, Socket.IO, Tailwind CSS, shadcn/ui, MSW, Playwright e Lighthouse.

1. Como apresentar o projeto

Kurio é um marketplace de NFTs com descoberta, conta e compra simuladas. O desafio pede uma experiência de produto completa sem depender de blockchain ou pagamentos reais. A decisão principal foi colocar os mocks na rede, para que a interface se comporte como um cliente de uma API: consultas podem atrasar, falhar ou mudar durante uma compra.

Comece a demonstração pelo catálogo, mostre a URL mudando com filtros e abra um NFT diretamente. Adicione ao carrinho, entre como Ana, aplique KURIO10 e conclua a compra. Depois mostre um cenário de falha e o histórico recuperando o mesmo pedido.

Os 15 frames do Figma foram exportados e revisados. Quatro retratos originais são usados no catálogo, com versões WebP locais. A família da fonte não foi resolvida pela inspeção: o painel apresenta a variável font/family. IBM Plex Mono é a substituição documentada. Apresente a identidade visual e explique as adaptações de formulários, rotas e acessibilidade, sem afirmar igualdade pixel a pixel.

2. Onde cada responsabilidade fica

React apresenta os estados. TypeScript estrito e Zod descrevem e validam os contratos. TanStack Router controla rotas, proteção e parâmetros de busca. TanStack Query mantém o cache remoto. Axios é o único cliente HTTP da aplicação. Tailwind e CSS expressam os tokens visuais. Componentes shadcn/Radix tratam composição, foco e diálogos.

MSW intercepta REST e WebSocket antes de a aplicação carregar. O banco da simulação está em uma camada própria e persiste as fixtures no navegador. Os componentes não contêm respostas alternativas para fingir sucesso quando a API falha.

A separação mais útil para explicar é: domínio define as regras, infraestrutura conecta, simulação responde e interface apresenta. Evite descrever a arquitetura como uma coleção de bibliotecas; explique o problema que cada uma resolve.

3. ETH e consistência dos valores

Uma unidade de ETH contém 10 elevado a 18 wei. JavaScript Number não representa todos esses valores com precisão. Por isso os contratos transportam strings decimais e os cálculos convertem para bigint em wei.

Exemplo importante: 0.100000000000000001 multiplicado por três precisa resultar em 0.300000000000000003. O utilitário de domínio e o teste verificam esse valor exato. O desconto trunca somente uma fração inferior a um wei. A apresentação troca o separador para vírgula e preserva as casas.

O frontend mostra subtotal, desconto, taxa e total devolvidos pela API. Na confirmação, utiliza o snapshot do pedido, nunca o catálogo atual. Uma alteração posterior de preço não reescreve um recibo antigo.

4. Compra idempotente e recuperação

Idempotência responde à pergunta: se o servidor criou a compra, mas o cliente não recebeu a resposta, como evitar comprar de novo? A aplicação grava uma chave UUID e o corpo da tentativa antes de enviar. Se ocorrer timeout, a chave não muda.

A simulação associa a chave ao usuário e ao conteúdo normalizado. Repetir a mesma tentativa retorna o mesmo pedido. Reutilizar a chave com outro conteúdo retorna conflito. A consulta da tentativa acontece antes da revalidação do estoque, para que um replay não seja confundido com uma compra nova.

O pedido nasce pendente e reserva estoque. Pode terminar confirmado ou recusado; ambos são terminais. Na recusa, o estoque reservado retorna. Na confirmação, apenas as quantidades compradas são removidas do carrinho atual. Isso preserva itens que o usuário adicionou enquanto o pedido aguardava.

Para demonstrar, selecione timeout-pedido, confirme a compra, aguarde a falha de resposta e recarregue. O botão Recuperar pedido retorna a mesma identificação. O histórico permite verificar que existe apenas um pedido.

5. Cotação, conexão e mudança durante a compra

O checkout tem duas etapas: revisar e confirmar. Revisar busca uma cotação atual. Confirmar envia seu identificador e a carteira selecionada. O servidor recalcula preço, disponibilidade, cupom e taxa e exige igualdade com a revisão.

A conexão da carteira também passa pela API simulada. O servidor registra qual carteira está conectada na sessão e verifica propriedade e rede. Um booleano enviado pela tela não é suficiente para autorizar a compra.

Mostre o cenário de preço alterado depois da revisão. O evento atualiza o carrinho por Socket.IO e invalida a cotação. O botão de confirmação fica bloqueado até nova revisão. O mesmo comportamento protege contra uma alteração que ocorra entre a revisão e a chegada da requisição ao servidor.

6. Tempo real sem perder a fonte de verdade

Socket.IO tem protocolo próprio; um WebSocket que apenas chama callbacks na tela não atenderia ao desafio. A solução usa socket.io-client, interceptação MSW e o binding publicado compatível com MSW 2. O transporte é WebSocket, com namespace padrão e eventos textuais.

Cada evento possui identidade, recurso e versão. O cliente acompanha a maior versão por recurso. Duplicatas e eventos antigos não reaplicam efeitos. Eventos privados também precisam corresponder ao usuário atual.

O evento informa que um recurso mudou; a API REST fornece o novo snapshot. Reconectar invalida as consultas ativas. Pedidos pendentes também têm polling de recuperação. Ao sair ou trocar de usuário, listeners são removidos e o socket é encerrado.

Uma particularidade conferida na implementação é que MSW 2 remove o prefixo /socket.io/ ao comparar o endpoint. Outra é que o cliente deve ser importado depois de iniciar MSW, porque captura o construtor WebSocket no carregamento.

7. Sessão, carrinho e isolamento

A sessão fictícia tem token e validade de uma hora. Rotas privadas consultam a API antes de abrir. O Axios trata 401, limpa a sessão e preserva a URL de retorno. Queries privadas incluem a identidade e o cache é cancelado e limpo na troca de conta.

O visitante tem identificador próprio. Login incorpora suas linhas ao carrinho da conta, usando a chave NFT mais edição e respeitando o estoque. Logout troca o identificador de visitante para que a conta anterior não apareça no carrinho público.

Tentativas e rascunhos de checkout ficam separados por usuário em sessionStorage. Uma correção importante foi aguardar a identidade antes de ler essa persistência. Caso contrário, um refresh tentaria recuperar uma chave sem usuário e perderia o contexto da tentativa.

As senhas são fictícias e seus hashes são persistidos. O hash simples com prefixo fixo atende à ausência de texto claro na demonstração, mas não deve ser defendido como segurança de produção. Um sistema real exige um backend de identidade, hashing lento com salt individual e proteção de sessão.

8. Cache, otimização e experiência

As queries ficam atuais por 15 segundos e são atualizadas ao focar a janela. Retries automáticos estão desativados para tornar falhas e novas tentativas explícitas. Cada leitura usa AbortSignal, o que impede respostas lentas da busca anterior de substituir a consulta atual.

Favoritar é a interação otimista: a tela responde antes da API, guarda um snapshot e restaura o estado se a mutation falhar. O refetch final reconcilia o cache. Não use atualização otimista para confirmar pagamento, porque a simulação é a única autoridade sobre o resultado.

Imagens e fontes são locais. Os quatro retratos foram convertidos para WebP sem alterar a composição. O carregamento inicial baixa a aplicação em paralelo à inicialização MSW; compra e conta usam importação sob demanda. O build inclui versões Brotli dos arquivos de texto, servidas no preview a clientes compatíveis. Skeletons reservam espaço e respeitam movimento reduzido. Foco visível, labels, mensagens associadas e foco preso em diálogos completam a experiência além do uso com mouse.

9. Como falar sobre os testes

Os testes Playwright observam a interface e os resultados das operações. Cada contexto começa isolado. Os cenários de rede são selecionados pelo painel e continuam atravessando MSW. Os eventos exercitam o socket real do cliente. As baselines de início, detalhe, carrinho e pagamento detectam regressões da própria aplicação.

Explique a diferença entre teste visual e fidelidade ao Figma: comparar duas screenshots da aplicação detecta mudanças, mas não prova que o layout original foi reproduzido. Explique também que Lighthouse fornece medições de laboratório, não desempenho garantido para todos os dispositivos.

O comando de auditoria mede início e detalhe três vezes em mobile e desktop, guarda cada HTML/JSON e calcula medianas. Performance, acessibilidade, boas práticas e SEO têm metas próprias. LCP, CLS e TBT ajudam a localizar a causa de uma pontuação menor.

Os resultados reais são anexados ao final do PDF a partir dos relatórios executados. Não invente percentuais nem diga que existe cobertura de um cenário sem indicar o teste correspondente.

10. Limites e evolução

A demonstração é estática e local ao navegador. Não há sincronização entre computadores nem lock distribuído entre várias abas. O backend real precisaria assumir persistência transacional, autenticação, autorização e idempotência durável.

O binding dos mocks não implementa toda a infraestrutura de um servidor Socket.IO. Não há blockchain real, extensão de carteira ou gateway financeiro, conforme o escopo do desafio. Referências de transação e exploração são explicitamente simuladas.

A fonte exata permanece sem confirmação. As diferenças de composição e as escolhas de UX estão em ARCHITECTURE.md: login em rota própria, revisão explícita e filtros acessíveis, por exemplo. Uma evolução técnica útil seria substituir a persistência local por serviço transacional e ampliar cenários concorrentes entre clientes independentes.

Na entrevista, demonstre o que funciona, explique a regra e indique a evidência. Para limites, descreva o efeito concreto e a próxima ação necessária. Isso é mais defensável do que apresentar a solução como se não tivesse restrições.

11. Evidências da entrega

Aplicação: https://kurio-jungle.vercel.app

Repositório: https://github.com/Mendes44/jungle-nft-marketplace

Última execução Playwright: 36 testes aprovados, 0 falhas inesperadas, 0 instáveis e 0 ignorados.

Consulte o relatório HTML para o nome de cada cenário, duração e projeto desktop/mobile. As baselines são da aplicação; não provam comparação integral com o Figma.

Na URL pública: 36 testes aprovados, 0 falhas inesperadas. Relatório em relatorios/publicacao/playwright-html/.

Lighthouse: medianas de três medições por página e perfil.

Página / perfil	Perf.	Acess.	Boas prát.	SEO
inicio / mobile	92	100	100	100
inicio / desktop	100	100	100	100
detalhe / mobile	91	100	100	100
detalhe / desktop	100	100	100	100

Página / perfil	LCP (ms)	CLS	TBT (ms)
inicio / mobile	3167	0.0000	51
inicio / desktop	665	0.0000	0
detalhe / mobile	3324	0.0000	36
detalhe / desktop	678	0.0000	0

Ambiente: win32 10.0.26200; 12th Gen Intel(R) Core(TM) i5-1235U; 16 GB; Node v24.18.1. Lighthouse 13.5.0.

Build Vite otimizado, MSW padrão, fontes e imagens locais, throttling padrão Lighthouse por perfil, navegador limpo por medição.

O que consultar durante a avaliação

Material	Finalidade
README.md	Instalar, executar e reproduzir cenários e falhas.
ARCHITECTURE.md	Sessão, precisão, concorrência, cache, eventos e limites.
docs/contratos.md	Campos, endpoints, status e eventos versionados.
docs/matriz-requisitos.md	Rastrear cada requisito e sua evidência.
docs/validacao.md	Resultados, URLs verificadas e pendências.
playwright-report/index.html	Relatório da suíte e evidências associadas.
relatorios/lighthouse/	Relatórios individuais e medianas reproduzíveis.

Referência visual: os 15 frames foram renderizados e revisados, e os quatro retratos originais estão incluídos. A fonte exata permanece sem confirmação: o Figma expõe a variável font/family. IBM Plex Mono é a substituição local; diferenças de composição estão documentadas em ARCHITECTURE.md.